

EBOOK · KHÓA HỌC 1 NGÀY · 5 SESSION

DevOps & Tự động hóa triển khai

Từ khái niệm đến pipeline CI/CD đầu tiên — thêm vào “vì sao”, đi kèm bài giải thực hành.

Plan · Code · Build · Test · Deploy · Monitor

Mario Education · phan@marioeducation.com

Mục lục

PHẦN I – NỘI DUNG

1. Giới thiệu về DevOps	3
2. Automation Tools: Ansible & Terraform	8
3. CI/CD – Tự động hóa triển khai	13
4. Container, Kubernetes & Giám sát	17
5. Nghề DevOps: Công việc, Chứng chỉ & Lộ trình	21

PHẦN II – BÀI GIẢI THỰC HÀNH

L1. Lab 1 – Dựng môi trường DevOps đầu tiên	25
L2. Lab 2 – Ansible playbook & Terraform	28
L3. Lab 3 – Pipeline CI/CD với GitHub Actions & Docker	32
L4. Lab 4 – Docker Compose, Kubernetes & Grafana	37

SESSION 1

Giới thiệu về DevOps

Mục tiêu bài học: Sau bài này bạn không chỉ *biết* DevOps là gì, mà hiểu **vì sao** nó tồn tại — để mọi công cụ học sau này (Docker, Kubernetes, CI/CD...) đều có chỗ đứng rõ ràng trong đầu bạn, thay vì là một mớ khái niệm rời rạc.

Vấn đề có trước, DevOps có sau

Trước khi nói DevOps là gì, hãy hỏi: **nó giải quyết nỗi đau nào?** Đây là câu hỏi quan trọng nhất của cả khóa học, vì mọi công cụ DevOps chỉ là câu trả lời cho một nỗi đau cụ thể.

Hãy hình dung một công ty phần mềm điển hình ~15 năm trước:

- **Đội Developer (Dev)** được thưởng khi **ra tính năng mới thật nhanh**. Mục tiêu của họ là *thay đổi*.
- **Đội Operations (Ops)** được thưởng khi **hệ thống chạy ổn định, không sập**. Mục tiêu của họ là *ổn định*.

Hai mục tiêu này **mâu thuẫn trực tiếp**. Mỗi lần Dev muốn deploy code mới, Ops lại thấy rủi ro. Kết quả là một hiện tượng kinh điển được gọi là **“bức tường hỗn loạn” (the wall of confusion)**: Dev “ném” code qua tường cho Ops, và khi có sự cố, hai bên đổ lỗi cho nhau.

Câu nói huyền thoại: “*It works on my machine*” — “Chạy tốt trên máy tôi mà!”. Dev nói câu này, Ops trả lời “nhưng nó không chạy trên production”, và không ai sai cả — vì hai môi trường vốn khác nhau.

Vì sao đây là vấn đề lớn? Không phải vì con người lười hay kém. Mà vì **cấu trúc tổ chức tạo ra động cơ đối nghịch**. Bạn không thể sửa vấn đề này bằng cách “cố gắng hơn” — bạn phải thay đổi *cách làm việc*. Đó chính là chỗ DevOps bước vào.

Vậy DevOps là gì?

DevOps = Development + Operations. Nhưng định nghĩa quan trọng nhất không nằm ở cái tên:

DevOps là một văn hóa và tập hợp thực hành nhằm xóa bỏ ranh giới giữa “xây” và “vận hành”, để giao phần mềm nhanh hơn mà vẫn ổn định hơn.

Chú ý ba từ khóa, và **vì sao** mỗi từ lại quan trọng:

1. **Văn hóa (culture)** — Đây là phần bị hiểu lầm nhiều nhất. Nhiều người nghĩ “DevOps = biết dùng Docker + Jenkins”. Sai. Bạn có thể mua mọi công cụ đắt tiền nhất mà vẫn *không* có DevOps, nếu Dev và Ops vẫn đổ lỗi cho nhau. Công cụ chỉ khuếch đại văn hóa — tốt hoặc xấu.
2. **Thực hành (practices)** — Những cách làm cụ thể: tự động hóa, tích hợp liên tục, giám sát... Bài học sau sẽ đi vào từng cái.
3. **Nhanh hơn và ổn định hơn** — Điểm phản trực giác nhất, sẽ nói ở phần dưới.

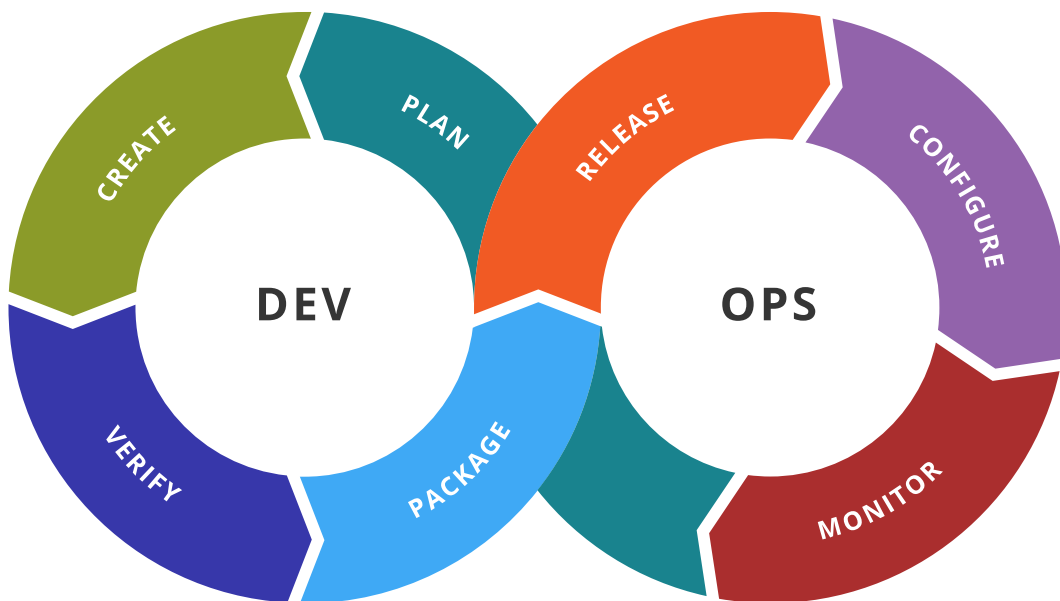
🔗 Vì sao "văn hóa" lại quan trọng hơn "công cụ"?

Vì công cụ có thể sao chép, văn hóa thì không. Đồ i thủ của bạn cũng dùng được Docker và Kubernetes chỉ sau một tuần. Nhưng một tổ chức mà Dev và Ops cùng chịu trách nhiệm, cùng trực sự cố, cùng nhìn một dashboard — đó là lợi thế cạnh tranh mà tiền không mua ngay được.

Vòng lặp DevOps: vì sao là vòng tròn vô tận?

Bạn sẽ thấy DevOps được vẽ thành một **vòng lặp vô hạn (∞)** gồm 8 giai đoạn:

Plan → **Code** → **Build** → **Test** → **Release** → **Deploy** → **Operate** → **Monitor** → (quay lại Plan)



Sơ đồ vòng đời và các nhóm công cụ DevOps (nguồn: Wikimedia Commons).

Vì sao là vòng tròn, không phải đường thẳng? Vì phần mềm không bao giờ “xong”. Trong mô hình cũ (Waterfall), ta hình dung: phân tích → code → test → giao hàng → *kết thúc*. Nhưng thực tế phần mềm là một sản phẩm *sống*: có người dùng, có lỗi phát sinh, có nhu cầu mới. Giai đoạn **Monitor** (giám sát) không phải điểm kết thúc — nó **feed ngược lại** giai đoạn **Plan**: dữ liệu vận hành thật cho bạn biết cần làm gì tiếp theo.

Đây là lý do vì sao ở Session 4 chúng ta sẽ nhấn mạnh: “*Deploy xong chưa phải là xong*”. Monitor khép kín vòng lặp.

Nghịch lý cô't lỗi: nhanh hơn *lại* an toàn hơn

Đây là ý tưởng khó tin nhấ't với người mới, nên cần giải thích kỹ.

Trực giác thông thường nói: “*Muố'n an toàn thì phải chậm, cẩn thận, ít thay đổi.*” Nên các công ty truyền thố'ng thường **release mỗi quý hoặc mỗi năm một lầ'n**, gom thật nhiề'u thay đổi vào một “big bang”.

Nhưng hãy nghĩ kỹ về **rủi ro của một lầ'n release**:

- Nế'u bạn gom **500 thay đổi** vào 1 lầ'n deploy mỗi 6 tháng, và có sự cô't → bạn phải dò trong **500 thay đổi** để tìm thủ phạm. Rollback thì mấ't luôn cả 500 thứ.
- Nế'u bạn deploy **1 thay đổi mỗi lầ'n, nhiề'u lầ'n một ngày**, và có sự cô't → thủ phạm gầ'n như chắ'c chắ'n là **thay đổi vừa rô'i**. Sửa hoặc rollback cực nhanh.

Vì sao nhỏ lại an toàn: Rủi ro không đế'n từ *tầ'n suấ't* thay đổi, mà từ *kích thước* mỗi lầ'n thay đổi. Chia nhỏ lô hàng (batch size) làm cho mỗi lầ'n deploy ít rủi ro hơn, dễ tìm lỗi hơn, dễ khôi phục hơn.

Nghiên cứu **DORA (DevOps Research and Assessment)** — khảo sát hàng chục nghìn tổ chức — chứng minh điề'u này bằ'ng số' liệu. Các đội “elite” so với đội yế'u nhấ't:

Chỉ số'	Đội elite vượt trội
Tầ'n suấ't deploy	973× thường xuyên hơn
Lead time (từ commit đế'n production)	6.570× ngắ'n hơn
Tỉ lệ thay đổi gây lỗi	3× thấ'p hơn
Thời gian khôi phục sự cô't	6.570× nhanh hơn

Điểm mắ'u chố't: đội elite **vừa nhanh hơn vừa ổn định hơn** — không phải đánh đổi. Tự động hóa chính là thứ khiế'n điề'u nghịch lý này thành hiện thực.

Vì sao cần nhiề'u môi trường (Dev / Test / Staging / Production)?

Một khái niệm nằ'n tảng: phầ'n mề'm chạy qua nhiề'u **môi trường** khác nhau.

- Development** — máy của lập trình viên, nơi code được viế't.
- Testing / QA** — nơi chạy kiểm thử tự động và thủ công.
- Staging** — bản sao *gầ'n giồ'ng hệt* production, để thử lầ'n cuố'i.
- Production** — môi trường thật, nơi người dùng thật truy cập.

Vì sao phải tách ra? Để cô lập rủi ro. Bạn không muốn một dòng code đang thử nghiệm làm sập hệ thống mà khách hàng đang dùng. Mỗi môi trường là một “lưới an toàn” trước khi tới production.

Nhưng tách môi trường lại đề ra chính vấn đề “*chạy trên máy tôi*”: nếu 4 môi trường này **khác nhau** (khác phiên bản thư viện, khác cấu hình, khác hệ điều hành), thì code chạy ở Dev chưa chắc chạy ở Production.

Đây chính là “vì sao” của container (Docker). Nếu ta đóng gói ứng dụng *cùng với* toàn bộ môi trường của nó vào một “hộp” chạy giống hệt nhau ở mọi nơi, thì vấn đề khác biệt môi trường biến mất. Ta sẽ học sâu ở Session 4 — nhưng bây giờ bạn đã hiểu lý do nó tồn tại.

Bản đồ công cụ: vì sao có từng nhóm

Người mới thường choáng vì có quá nhiều công cụ DevOps. Bí quyết: **đừng học công cụ, hãy học nhóm vấn đề mà công cụ giải quyết.** Mỗi giai đoạn trong vòng lặp có một nhóm công cụ:

Nhóm vấn đề	Công cụ tiêu biểu	Vì sao cần
Quản lý mã nguồn	Git , GitHub, GitLab	Nhiều người sửa cùng codebase mà không giẫm chân nhau; lưu lịch sử để rollback
CI/CD (tự động build–test–deploy)	Jenkins , GitHub Actions, GitLab CI	Biến quy trình thủ công dễ sai thành pipeline tự động, lặp lại được
Đóng gói ứng dụng	Docker	Giải quyết “chạy trên máy tôi” bằng container đồng nhất
Điều phối container	Kubernetes	Chạy hàng trăm container, tự phục hồi, tự scale
Quản lý cấu hình	Ansible	Cài đặt & cấu hình hàng loạt máy chủ mà không làm thủ công
Hạ tầng như mã (IaC)	Terraform	Tạo/hủy hạ tầng cloud bằng code, tái lập được
Giám sát & cảnh báo	Prometheus , Grafana	Biết hệ thống đang sống hay chết, phát hiện sự cố sớm
Trợ lý AI	ChatGPT , Claude, AI agent	Viết script, review cấu hình, debug nhanh hơn



Cả khóa học này là hành trình đi qua từng nhóm — mỗi session lấp thêm một mảnh ghép cho tới khi bạn có bức tranh hoàn chỉnh.

Vì sao “tự động hóa” là trái tim của DevOps

Nếu phải rút gọn DevOps thành một chữ, đó là **tự động hóa (automation)**. Vì sao?

1. **Con người làm việc lặp lại thì hay sai.** Một checklist deploy 40 bước làm bằng tay, kiểu gì cũng có ngày quên bước 27. Máy thì không quên.
2. **Tự động hóa biến kiến thức thành code.** Thay vì “anh A là người duy nhất biết cách deploy”, quy trình nằm trong script mà cả đội đọc được, sửa được, học được.
3. **Tự động hóa mở khóa cho tốc độ và an toàn.** Chính vì deploy được tự động hóa và đáng tin cậy, ta mới dám deploy nhiều lần một ngày (nhắc lại nghịch lý ở trên).

Nguyên tắc vàng: Bất cứ việc gì bạn làm bằng tay hơn hai lần, hãy nghĩ đến việc tự động hóa nó.

AI trong DevOps (thực hành mở đầu)

Trong buổi học, hãy thử dùng **ChatGPT / Claude** như một trợ lý và **so sánh** với hiểu biết của bạn:

- “So sánh Jenkins vs GitLab CI vs GitHub Actions cho một team 5 người.”
- “Giải thích DevOps cho người mới bằng một ví dụ đời thường.”

Nhưng luôn giữ tư duy phản biện: **AI trả lời rất trôi chảy, không có nghĩa là đúng.** Kỹ năng quan trọng của kỹ sư DevOps thời AI không phải là “hỏi được AI”, mà là **kiểm chứng được câu trả lời của AI.** Đây là chủ đề xuyên suốt khóa học.

Tóm tắt: những “vì sao” cần nhớ

- **Vì sao có DevOps?** Vì Dev (muốn thay đổi) và Ops (muốn ổn định) có động cơ đối nghịch, tạo ra “bức tường hỗn loạn”.
- **Vì sao là văn hóa, không phải công cụ?** Vì công cụ sao chép được, văn hóa cộng tác thì không.
- **Vì sao nhanh lại an toàn?** Vì chia nhỏ thay đổi làm giảm rủi ro và thời gian khôi phục mỗi lần deploy.
- **Vì sao cần nhiều môi trường + container?** Để cô lập rủi ro mà vẫn giữ các môi trường đồng nhất.
- **Vì sao tự động hóa là cốt lõi?** Vì nó loại bỏ lỗi con người và mở khóa cho cả tốc độ lẫn độ tin cậy.

👉 **Thực hành:** Xem [Bài giải Lab 1 — Dựng môi trường DevOps đầu tiên](#) để bắt tay vào cài đặt môi trường dev + test.

SESSION 2

Automation Tools: Ansible & Terraform

Mục tiêu bài học: Hiểu **vì sao** ta cần tự động hóa hạ tầng, phân biệt được hai bài toán khác nhau (cấu hình vs tạo lập hạ tầng), và biết công cụ nào giải quyết bài toán nào.

Vì sao làm thủ công không thể mở rộng?

Hãy tưởng tượng bạn quản lý **3 server**. Cài Nginx lên cả ba bằng tay: SSH vào từng máy, gõ `apt install nginx`, sửa file config, khởi động dịch vụ. Mất 15 phút, chấp nhận được.

Giờ tăng lên **300 server**. Vấn đề bùng nổ:

1. **Không thể lặp lại chính xác.** Bạn gõ tay 300 lần, kiểu gì cũng có máy bị gõ sai, quên một bước, hoặc cài nhầm phiên bản.
2. **Config drift (cấu hình trôi dạt).** Theo thời gian, mỗi server bị sửa vặt một ít — máy này ai đó tăng giới hạn RAM, máy kia ai đó tắt một dịch vụ. Sau 6 tháng, không còn hai server nào giống nhau. Đây gọi là “**snowflake server**” — mỗi máy độc nhất như một bông tuyết, và *không ai dám đụng vào* vì sợ hỏng.
3. **Không có nguồn sự thật.** Muốn biết server đang cấu hình thế nào? Phải SSH vào xem. Không có tài liệu nào đáng tin, vì tài liệu luôn lỗi thời so với thực tế.

Vì sao "snowflake server" nguy hiểm?

Khi một snowflake server chết, bạn **không thể tái tạo nó** — vì không ai biết chính xác nó đã được cấu hình thế nào qua nhiều năm chỉnh sửa thủ công. Cả hệ thống phụ thuộc vào một cái máy mà không ai dám khởi động lại. Tự động hóa cấu hình sinh ra chính là để diệt tận gốc vấn đề này.

Ansible — vì sao nó thắng trong quản lý cấu hình

Ansible giải quyết bài toán: “*làm sao cấu hình hàng loạt server một cách nhất quán, lặp lại được?*”
Ba đặc tính khiến nó phổ biến, và **vì sao** mỗi cái quan trọng:

1. Agentless — vì sao đây là lợi thế lớn

Nhiều công cụ đối thủ (như Puppet, Chef) yêu cầu cài một “agent” (phần mềm nền) trên *mọi* máy đích. Nghe thì nhỏ, nhưng: ai cài agent lên 300 máy? Ai cập nhật agent khi có bản vá bảo mật? Agent chết thì sao?

Ansible **không cần agent** — nó chỉ dùng **SSH** (thứ mà mọi server Linux đã có sẵn). Bạn chạy Ansible từ một máy điều khiển (control node), nó SSH vào các máy đích và thực thi. Không cài gì thêm lên máy đích. Đây là lý do Ansible dễ bắt đầu hơn hẳn.



2. Idempotent — vì sao “chạy lại nhiều lần vẫn an toàn” lại quan trọng

Idempotent nghĩa là: chạy playbook một lần hay mười lần, kết quả cuối cùng vẫn như nhau.

Vì sao cần? Hãy so sánh:

- **Script thường (imperative):** “Chạy lệnh `apt install nginx`”. Nếu Nginx đã cài rồi mà bạn chạy lại, có thể gây lỗi hoặc cài đè.
- **Ansible (declarative):** “Đảm bảo Nginx ở trạng thái `present`”. Ansible tự kiểm tra: nếu Nginx đã có, nó **không làm gì cả**; nếu chưa có, nó cài. Chạy bao nhiêu lần cũng an toàn.

Vì sao điều này thay đổi cuộc chơi: Bạn có thể chạy playbook **định kỳ** để “ép” mọi server về đúng trạng thái mong muốn — tự động sửa mọi config drift. Server bị ai đó sửa lung tung? Chạy lại playbook, nó về đúng chuẩn.

3. Playbook — mô tả trạng thái mong muốn bằng YAML

Bạn không viết “các bước để làm”, bạn viết “**trạng thái bạn muốn**”. Ví dụ:

```
- hosts: web
  become: yes          # chạy với quyền sudo
  tasks:
    - name: Cài đặt Nginx
      apt:
        name: nginx
        state: present  # "hãy đảm bảo nginx được cài"

    - name: Khởi động và bật Nginx tự chạy khi boot
      service:
        name: nginx
        state: started
        enabled: yes
```

File này là **tài liệu số** về cấu hình server: dễ đọc, nằm trong Git, cả đội cùng review được. Không còn cảnh “chỉ anh A biết cách cấu hình”.

Terraform & Infrastructure as Code — một bài toán khác

Đây là điểm nhiều người mới nhầm lẫn, nên cần tách bạch rõ.

Ansible trả lời: “Server này nên được **cấu hình** thế nào?” (cài phần mềm gì, config ra sao).

Nhưng có một câu hỏi đặt **trước** đó: “Server này từ đâu **ra**?” Ai tạo ra máy ảo, mạng, database, load balancer trên cloud?

Terraform trả lời câu hỏi này. Nó thuộc nhóm **Infrastructure as Code (IaC)** — quản lý hạ tầng (không phải phần mềm bên trong) bằng mã.



Vì sao “hạ tầng như mã” lại quan trọng đến vậy?

Trước IaC, người ta tạo hạ tầng bằng cách **bấm chuột** trong giao diện web của AWS/Azure: tạo máy ảo, cấu hình mạng, mở cổng firewall... Vấn đề y hệt như cấu hình thủ công, nhưng ở tầng hạ tầng:

- **Không tái lập được.** Sếp bảo “tạo môi trường staging giống hệt production”. Bạn nhớ nổi 200 cú click đã tạo ra production không?
- **Không có lịch sử.** Ai đó mở cổng 22 ra Internet lúc nào? Không log, không biết.
- **Không review được.** Thay đổi hạ tầng là bấm nút, không ai duyệt trước.

Với Terraform, hạ tầng trở thành **code**:

```
resource "docker_container" "web" {
  name = "web-server"
  image = "nginx:latest"
  ports {
    internal = 80
    external = 8080
  }
}
```

Và bạn nhận được **mọi lợi ích của code**:

- **Version control:** hạ tầng nằm trong Git, có lịch sử đầy đủ, rollback được.
- **Code review:** thay đổi hạ tầng phải qua pull request, có người duyệt.
- **Tái lập:** cùng một file `.tf` tạo ra môi trường giống hệt nhau ở dev, staging, production.

Vòng đời Terraform: vì sao có bước `plan`

Terraform hoạt động theo 4 lệnh chính:

```
terraform init    # tải plugin, khởi tạo thư mục làm việc
terraform plan    # XEM TRƯỚC những gì sẽ thay đổi (chưa làm gì cả)
terraform apply   # thực sự tạo/sửa hạ tầng
terraform destroy # xóa toàn bộ hạ tầng đã tạo
```

Bước **plan** là điểm sáng giá nhất. **Vì sao?** Vì hạ tầng rất nguy hiểm để “thử rồi sửa” — xóa nhầm một database production là thảm họa. **terraform plan** cho bạn xem *chính xác* Terraform sẽ tạo gì, sửa gì, xóa gì — **trước khi** bất cứ điều gì xảy ra. Đây là tấm lưới an toàn.

Terraform nhớ trạng thái hạ tầng thực tế trong một **state file**, nên nó luôn biết khác biệt giữa “cái đang có” và “cái bạn muốn”, rồi chỉ thực hiện đúng phần chênh lệch.

Ansible vs Terraform: vì sao dùng cả hai?

Câu hỏi kinh điển của người mới: “Hai cái này làm cùng một việc à? Chọn cái nào?”

Trả lời: **chúng giải quyết hai bài toán khác nhau, và thực tế bạn dùng cả hai cùng nhau.**

	Terraform	Ansible
Bài toán	Provisioning — tạo hạ tầng	Configuration — cấu hình hạ tầng
Ví dụ	Tạo 3 máy ảo, 1 mạng, 1 database trên AWS	Cài Nginx + PHP + cấu hình app lên 3 máy đó
Tư duy	Khai báo (declarative)	Chủ yếu khai báo
Vòng đời	Quản lý cả tạo lẫn hủy hạ tầng	Đưa máy về trạng thái mong muốn

Quy trình thực tế:

1. **Terraform** dựng lên các máy chủ trống (provisioning).
2. **Ansible** cài đặt và cấu hình phần mềm bên trong các máy đó (configuration).

Ví như xây nhà: Terraform xây phần thô (móng, tường, mái), Ansible lo phần hoàn thiện (điện, nước, sơn, nội thất).

💡 Vì sao khai báo (declarative) tốt hơn ra lệnh (imperative)?

Với tư duy *ra lệnh*, bạn phải mô tả **mọi bước** và xử lý mọi trường hợp (“nếu đã cài thì bỏ qua, nếu chưa thì cài, nếu sai phiên bản thì gỡ...”). Với tư duy *khai báo*, bạn chỉ nói **kết quả mong muốn**, công cụ tự tính cách đạt được. Ít code hơn, ít lỗi hơn, và luôn hội tụ về đúng trạng thái dù xuất phát từ đâu.

AI trong hạ tầng (thực hành nâng cao)

Trong buổi học, ta thử một quy trình thời AI:

1. Cài một **AI coding agent** (ví dụ Antigraity), yêu cầu nó **dựng một Ansible server** và sinh playbook.

2. Nâng cao: chạy **Kubespray** (bộ Ansible playbook để cài Kubernetes) — khi gặp lỗi, dán log cho AI **debug**.

Vì sao bài này quan trọng: AI có thể sinh cấu hình rất nhanh, nhưng cũng sai rất tự tin. Kỹ năng cốt lõi là “**AI viết – người kiểm tra**”: bạn phải đủ hiểu bản chất (những gì học ở trên) để biết khi nào AI sai. Công cụ AI khuếch đại năng lực của người đã hiểu, chứ không thay thế sự hiểu.

Tóm tắt: những “vì sao” cần nhớ

- **Vì sao cần tự động hóa hạ tầng?** Vì thủ công không lặp lại được, sinh ra config drift và snowflake server.
- **Vì sao Ansible agentless?** Vì không phải cài/bảo trì agent trên hàng trăm máy — chỉ cần SSH.
- **Vì sao cần idempotent?** Để chạy lại an toàn, tự sửa config drift, ép hệ thống về đúng chuẩn.
- **Vì sao IaC/Terraform?** Để hạ tầng có version control, review được, tái lập được — như code.
- **Vì sao có bước `plan`?** Để xem trước thay đổi nguy hiểm trước khi thực thi.
- **Vì sao dùng cả hai?** Terraform *tạo* hạ tầng, Ansible *cấu hình* nó — hai bài toán khác nhau.

👉 **Thực hành:** Bài giải Lab 2 — Ansible playbook & Terraform

SESSION 3

CI/CD — Tự động hóa triển khai ứng dụng

Mục tiêu bài học: Hiểu **vì sao** CI/CD tồn tại (nó chữa nỗi đau gì), phân biệt được ba khái niệm CI / Delivery / Deployment, và nắm được vì sao các chiến lược deploy khác nhau lại cần thiết.

Vì sao có Continuous Integration? Câu chuyện “integration hell”

Quay lại thời chưa có CI. Một team 5 lập trình viên, mỗi người làm một tính năng trên một nhánh (branch) riêng, trong **2–3 tuần**. Ai cũng code hăng say trên máy mình.

Đến ngày gộp code (merge) lại với nhau — **thảm họa**:

- Người A đổi tên một hàm mà người B đang dùng.
- Người C và người D sửa cùng một file theo hai hướng mâu thuẫn.
- Code của mỗi người chạy tốt riêng lẻ, nhưng ghép lại thì vỡ tung.

Việc gộp mã **tuyệt vời**, đầy lỗi, ai cũng căng thẳng. Hiện tượng này có tên: **“integration hell”** (địa ngục tích hợp). Càng để lâu mới gộp, càng nhiều xung đột tích tụ, càng đau.

Vì sao để càng lâu càng đau?

Vì xung đột tích lũy theo cấp số nhân với thời gian và số dòng thay đổi. Gộp 2 tuần code của 5 người = hàng nghìn dòng va chạm nhau cùng lúc. Gộp mỗi ngày = vài chục dòng, xung đột (nếu có) nhỏ và dễ xử lý. **Giải pháp không phải “gộp cẩn thận hơn” mà là “gộp thường xuyên hơn”.**

Continuous Integration (CI) chính là kỷ luật này: mỗi lập trình viên **tích hợp code vào nhánh chung nhiều lần mỗi ngày**, và **mỗi lần tích hợp được máy tự động build + chạy test** để phát hiện xung đột ngay lập tức.

Thay vì biết mình gây lỗi sau 2 tuần, bạn biết sau **5 phút** — khi vẫn đề còn nhỏ và còn nhớ mình vừa sửa gì.

Vì sao có Continuous Delivery/Deployment?

CI lo phần “code gộp lại có build & test được không”. Nhưng còn một khoảng cách nữa: từ “code đã test xong” đến “chạy trên máy người dùng thật”.

Trong nhiều tổ chức, khoảng cách này vẫn thủ công và đáng sợ: một đợt “release” là sự kiện lớn, làm vào cuối tuần, cả đội thức đêm, cả ưu nguyện không có gì hỏng.

Continuous Delivery/Deployment áp dụng đúng nghịch lý đã học ở Session 1 — *nhỏ và thường xuyên thì an toàn hơn lớn và hiếm hoi* — vào khâu triển khai: tự động hóa để deploy trở thành việc **nhàm chán, hàng ngày**, thay vì sự kiện kinh hoàng.

Ba cấp độ: CI vs Delivery vs Deployment

Ba từ này hay bị dùng lẫn lộn. Phân biệt rõ:

Cấp độ	Tự động tới đâu	Ranh giới
Continuous Integration	Mỗi push → tự động build + test	Dừng ở việc kiểm tra code
Continuous Delivery	Trên nữa: luôn có bản sẵn sàng release , nhưng bấm nút mới lên production	Có một cửa phê duyệt của con người
Continuous Deployment	Cao nhất: qua hết test là tự động lên production , không cần con người	Không có cửa thủ công nào

```

CI:           Code → Build → Test
Delivery:     Code → Build → Test → Release → [người bấm nút] → Production
Deployment:   Code → Build → Test → Release → Deploy (tự động hoàn toàn)
  
```

Vì sao đa số bắt đầu từ CI: Bạn không nhảy thẳng lên Continuous Deployment. Hầu hết team đi tuấn tự: làm CI vững trước (test tự động đáng tin), rồi lên Delivery (deploy một nút bấm), cuối cùng mới Deployment (bỏ luôn nút bấm) khi đã đủ tin vào bộ test của mình.

Giải phẫu một pipeline: vì sao có từng stage

Một **pipeline** là chuỗi các giai đoạn (stage) tự động chạy khi có code mới. Ví dụ điển hình:

Trigger → Build → Test → Deploy

- **Trigger (kích hoạt):** cái gì làm pipeline chạy? Thường là **push** code, mở **merge request**, hoặc theo lịch. *Vì sao:* để pipeline chạy tự động, không cần ai bấm.
- **Build:** biên dịch code, đóng gói thành artifact (ví dụ Docker image). *Vì sao:* đảm bảo code thực sự build được, và tạo ra “sản phẩm” nhất quán để đem đi deploy.
- **Test:** chạy unit test, kiểm tra bảo mật, integration test. *Vì sao:* đây là **cửa chốt lượng** — code không qua test thì không được đi tiếp. Đây là thứ chặn bug trước khi tới người dùng.
- **Deploy:** đẩy artifact lên môi trường (staging rồi production). *Vì sao:* tự động hóa khâu nguy hiểm nhất, loại bỏ lỗi con người khi deploy tay.

Vài thuật ngữ bạn sẽ gặp:

- **Stage / Job / Step:** pipeline chia thành giai đoạn → công việc → từng bước lệnh.
- **Runner:** máy (hoặc container) thực sự thực thi các job.
- **Artifact:** sản phẩm pipeline sinh ra — Docker image, file build, báo cáo test.



Công cụ phổ biến: **Jenkins** (mạnh, linh hoạt, tự host), **GitHub Actions** (tích hợp sẵn GitHub, dễ bắt đầu), **GitLab CI** (tích hợp sẵn GitLab).

Git workflow: vì sao cần chiến lược nhánh

Pipeline chạy dựa trên Git, nên cách bạn tổ chức nhánh ảnh hưởng trực tiếp tới CI/CD.

- **Trunk-based development:** mọi người làm trên nhánh ngắn (sống vài giờ đến 1–2 ngày), merge vào `main` liên tục. Vì sao hợp CI/CD: nhánh ngắn = ít xung đột, tích hợp thật sự “liên tục”.
- **Git Flow:** nhiều nhánh dài (develop, release, hotfix...). Vì sao ít hợp CI/CD: nhánh sống lâu tái tạo lại chính “integration hell” mà CI muốn diệt. Phù hợp hơn với sản phẩm release theo phiên bản (như phần mềm đóng gói).

Quy trình chuẩn: `feature branch` → mở **pull request** → **review** (đồng nghiệp duyệt) → merge vào `main` → pipeline tự chạy. Bước review là một **cửa chất lượng của con người**, bổ sung cho các cửa tự động.

Chiến lược triển khai: vì sao cần deploy “không downtime”

Khi deploy phiên bản mới, câu hỏi là: *làm sao thay code cũ bằng code mới mà người dùng không bị gián đoạn, và nếu bản mới lỗi thì quay lại nhanh?*

Có ba chiến lược chính, mỗi cái là một sự đánh đổi khác nhau:

Rolling Update (cập nhật tuần hoàn)

Thay dần từng phần: tắt 1 instance cũ, bật 1 instance mới, lặp lại cho tới hết. Đây là mặc định của Kubernetes.

- **Ưu:** không tốn thêm tài nguyên.
- **Nhược:** trong lúc tuần hoàn, hai phiên bản chạy song song (có thể gây lỗi nếu không tương thích); rollback chậm.

Blue-Green

Duy trì **hai môi trường giống hệt nhau**: “blue” (đang chạy) và “green” (bản mới). Deploy lên green, test xong thì **chuyển toàn bộ traffic** sang green trong một nhịp.

- **Ưu:** rollback **tức thì** — chỉ cần chuyển traffic ngược lại blue.
- **Nhược:** tốn **gấp đôi tài nguyên** (phải nuôi hai môi trường).

Canary Release

Đưa bản mới cho **một nhóm nhỏ** người dùng trước (5–10%), quan sát; ổn thì mở rộng dần tới 100%. (Tên “canary” — chim hoàng yến — lấy từ việc thợ mỏ xưa mang chim vào hầm để phát hiện khí độc sớm.)

- **Ưu:** an toàn nhất — phát hiện lỗi trên người dùng thật với thiệt hại giới hạn.
- **Nhược:** phức tạp — cần cơ chế chia traffic và giám sát tinh vi.

🔗 Vì sao không có chiến lược “tốt nhất”?

Vì mỗi cái đánh đổi giữa **chi phí tài nguyên**, **độ phức tạp** và **độ an toàn** khác nhau. Một blog cá nhân dùng rolling update là đủ. Một ngân hàng đổi hệ thống thanh toán sẽ muốn canary để giới hạn rủi ro. Chọn theo mức rủi ro bạn chấp nhận được và tài nguyên bạn có — chứ không phải theo cái “xịn nhất”.

AI trong CI/CD (thực hành)

- Dán file pipeline (`.yaml`) vào ChatGPT/Claude: “Review pipeline này, chỉ ra lỗi hỏng bảo mật và điểm có thể tối ưu.”
- Nhờ AI **sinh unit test** cho ứng dụng mẫu, rồi thêm vào stage test.

Vì sao hữu ích mà vẫn phải cẩn thận: AI rất giỏi phát hiện các lỗi phổ biến (lộ secret trong pipeline, thiếu bước test). Nhưng nó không hiểu ngữ cảnh riêng hệ thống của bạn. Dùng AI như một *reviewer thứ hai*, không phải người quyết định cuối.

Tóm tắt: những “vì sao” cần nhớ

- **Vì sao có CI?** Để diệt “integration hell” — tích hợp thường xuyên thì xung đột nhỏ và dễ sửa.
- **Vì sao có CD?** Để biến deploy từ sự kiện đáng sợ thành việc nhàm chán, hàng ngày.
- **Vì sao phân biệt CI/Delivery/Deployment?** Vì chúng là ba mức tự động hóa khác nhau; hầu hết team tiến dần qua từng mức.
- **Vì sao có từng stage trong pipeline?** Mỗi stage là một cửa: build (code chạy được?), test (chất lượng?), deploy (đưa an toàn lên production).
- **Vì sao cần chiến lược deploy?** Để đổi phiên bản không gây gián đoạn và rollback nhanh khi lỗi.

👉 **Thực hành:** Bài giải Lab 3 — Pipeline CI/CD với GitHub Actions & Docker

SESSION 4

Container, Kubernetes & Giám sát hệ thống

Mục tiêu bài học: Hiểu **vì sao** container ra đời, **vì sao** cần một hệ điều hành phổ biến như Kubernetes, và **vì sao** giám sát là mất xích không thể thiếu — chứ không chỉ học cú pháp lệnh.

Vì sao cần container? Chuyện “chạy trên máy tôi”

Nhớ lại Session 1: vấn đề lớn nhất là các môi trường (dev, test, production) **khác nhau**, nên code chạy chỗ này lại hỏng chỗ kia.

Trước container, người ta thử giải bằng **máy ảo (VM)**: đóng gói cả một hệ điều hành đầy đủ. Nó giải quyết được vấn đề đồng nhất, nhưng **quá nặng**:

- Mỗi VM chứa cả một OS riêng → tốn hàng GB, khởi động mất vài phút.
- Chạy 10 ứng dụng nhỏ = 10 OS đầy đủ → lãng phí khủng khiếp.

Container là bước tiến hóa: nó đóng gói ứng dụng *cùng mọi thứ nó cần để chạy* (thư viện, cấu hình), nhưng **dùng chung nhân (kernel)** của hệ điều hành **host** thay vì mang theo cả một OS.



Kết quả:

- **Nhẹ:** tính bằng MB thay vì GB; khởi động trong **mili-giây**.
- **Đồng nhất:** container chạy **giống hệt nhau** ở mọi nơi — máy dev, máy test, production. Đây chính là lời giải triệt để cho “It works on my machine”.

🎯 Vì sao “đóng gói kèm môi trường” lại là chìa khóa?

Vì nó lật ngược vấn đề. Thay vì cố làm mọi môi trường giống nhau (bất khả thi), ta **mang môi trường đi cùng ứng dụng**. Container chứa sẵn đúng phiên bản Python, đúng thư viện, đúng config — nên môi trường host là gì không còn quan trọng. Ứng dụng luôn thấy đúng thứ nó cần.

Image vs Container, và Dockerfile

- **Image (ảnh):** khuôn mẫu bất biến, chứa ứng dụng + môi trường. Ví như “bản thiết kế nhà”.
- **Container:** một instance đang chạy được tạo từ image. Ví như “ngôi nhà xây từ bản thiết kế”. Một image → tạo được nhiều container giống hệt.

- **Dockerfile:** công thức để build image, nằm trong Git cùng source code.

Vài best practice và **vì sao**:

- **Image nhỏ (dùng base `alpine`):** tải nhanh hơn, ít lỗi hỏng bảo mật hơn.
- **Tận dụng layer cache:** Docker build theo tầng, tầng không đổi được dùng lại → build nhanh hơn nhiều.
- **Không chạy bằng user `root`:** giới hạn thiệt hại nếu container bị tấn công.

Docker Compose: vì sao cần

Ứng dụng thật phức tạp khi chỉ có một phần. Ví dụ: một web app + một database + một cache. Chạy tay từng container, nối mạng giữa chúng — mệt mỏi.

Docker Compose mô tả toàn bộ ứng dụng nhiều thành phần trong **một file YAML**:

```
services:
  web:
    build: .
    ports:
      - "80:3000"
  db:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD: secret
```

Một lệnh `docker compose up` dựng lên tất cả, tự nối mạng. *Vì sao hữu ích:* cả môi trường phức tạp trở thành một file version-controlled, ai clone về cũng chạy được y hệt.

Vì sao cần Kubernetes?

Docker Compose ổn cho một máy. Nhưng production thật cần chạy hàng trăm container trên **nhiều máy**, và phải đối mặt với những câu hỏi khó:

- Một container chết lúc 2 giờ sáng thì ai bật lại?
- Lượng truy cập tăng gấp 10 vào giờ cao điểm thì ai thêm container, giờ vắng thì ai bớt đi?
- Deploy phiên bản mới mà không downtime thì làm sao?
- Traffic phân bổ đều giữa các container thế nào?

Làm thủ công những việc này ở quy mô lớn là bất khả thi. **Kubernetes (K8s)** là hệ **điều phối container (orchestration)** — nó tự động lo tất cả.



Các khái niệm lõi (và vì sao có chúng)

- **Pod:** đơn vị nhỏ nhất K8s quản lý, bọc 1 (hoặc vài) container. *Vì sao không quản container trực tiếp:* Pod thêm một lớp trừu tượng để nhóm các container gắn bó chặt với nhau.
- **Deployment:** quản lý số bản sao (replica) của Pod, lo rolling update và **tự phục hồi**. *Vì sao quan trọng nhất:* bạn khai báo “tôi muốn luôn có 3 bản chạy”, Deployment đảm bảo điều đó — pod chết thì nó tạo pod mới thay thế.
- **Service:** cung cấp một địa chỉ truy cập **ổn định** + cân bằng tải cho nhóm Pod. *Vì sao cần:* Pod sinh ra và chết liên tục, địa chỉ thay đổi; Service là “cửa trước” cố định để truy cập chúng.
- **Namespace:** phân vùng tài nguyên theo team hoặc môi trường.

Vì sao “self-healing” thay đổi cách ta vận hành

Điểm mạnh cốt lõi của K8s: bạn **khai báo trạng thái mong muốn** (declarative, giống Terraform/Ansible), và K8s **liên tục ép hệ thống về trạng thái đó**.

Bạn nói “tôi muốn 3 replica”. Một pod chết? K8s tự tạo pod mới. Một node (máy chủ) sập? K8s dời pod sang node khác. Bạn không cần trực đêm để bật lại dịch vụ — hệ thống tự chữa. Đây là lý do K8s trở thành nền tảng vận hành hiện đại.

Vì sao “deploy xong chưa phải là xong”?

Đây có lẽ là bài học bị bỏ quên nhiều nhất của người mới. Deploy thành công **không** có nghĩa là công việc kết thúc. Nhớ lại vòng lặp DevOps ở Session 1: **Monitor** khép kín vòng tròn và feed ngược lại **Plan**.

Vì sao giám sát (monitoring) là bắt buộc, không phải tùy chọn:

- Hệ thống chạy trên production sẽ gặp những vấn đề **không bao giờ xuất hiện** ở môi trường test: tải thật, dữ liệu thật, hành vi người dùng thật.
- Nếu không đo lường, bạn **mù**. Hệ thống chậm dần, sập hết bộ nhớ, tỉ lệ lỗi tăng — bạn không biết gì cho tới khi khách hàng phàn nàn (hoặc bỏ đi).



Luồng giám sát điển hình

Ứng dụng → Prometheus → Grafana → Alert

- Ứng dụng/Server phơi ra các **metrics** (số liệu: CPU, RAM, số request, độ trễ).
- **Prometheus** thu thập và lưu các metrics này theo thời gian.
- **Grafana** vẽ chúng thành **dashboard** trực quan.
- **Alert** tự động cảnh báo khi một chỉ số vượt ngưỡng (ví dụ tỉ lệ lỗi > 5%).

SLI và SLO: vì sao cần đo lường có mục tiêu

- **SLI (Service Level Indicator):** một chỉ số phản ánh **trải nghiệm thật** của người dùng — độ trễ, tỉ lệ lỗi, uptime.
- **SLO (Service Level Objective):** một **cam kết cụ thể** về SLI đó, ví dụ “99,9% request thành công trong tháng”.

Vì sao SLO mạnh: nó biến “chất lượng” mơ hồ thành con số quyết định được. Khi bạn tiêu hết “ngân sách lỗi” (error budget) cho phép của SLO, đó là tín hiệu khách quan để **dừng ra tính năng mới, dồn sức vào ổn định**. Không còn tranh cãi cảm tính “nhanh hơn hay ổn hơn” — số liệu quyết định.

Ngoài metrics, **logging tập trung** (gom log mọi service về một chỗ như ELK/Loki) giúp truy vết sự cố nhanh, thay vì SSH vào từng máy mò log.

Quy tắc vàng: Nếu bạn không đo được nó, bạn không vận hành được nó. Giám sát là thứ biến “deploy và cầu nguyện” thành “deploy và quan sát”.

AI trong vận hành (thực hành)

- Dán log lỗi của pod (`CrashLoopBackOff` , `ImagePullBackOff` ...) vào AI và yêu cầu chẩn đoán nguyên nhân.
- So sánh với cách debug thủ công: `kubectl describe pod <tên>` và `kubectl logs <tên>` .

Vì sao đáng thử: các lỗi K8s phổ biến có mẫu rõ ràng, AI chẩn đoán rất nhanh. Nhưng hãy kiểm chứng bằng lệnh thật — AI đoán dựa trên log bạn đưa, nếu bạn đưa thiếu thông tin, nó đoán sai.

Tóm tắt: những “vì sao” cần nhớ

- **Vì sao container thắng VM?** Nhẹ hơn nhiều (dùng chung kernel) mà vẫn đồng nhất môi trường.
- **Vì sao container giải quyết “chạy trên máy tôi”?** Vì nó mang môi trường đi cùng ứng dụng.
- **Vì sao cần Kubernetes?** Để tự động điều khiển hàng trăm container: self-healing, auto-scale, load balancing, deploy không downtime.
- **Vì sao self-healing quan trọng?** Vì bạn khai báo trạng thái mong muốn, K8s tự giữ hệ thống ở đó — không cần can thiệp tay.
- **Vì sao deploy xong chưa phải xong?** Vì production sinh ra vấn đề mà test không có; không giám sát = mù.
- **Vì sao cần SLO?** Để biến chất lượng mơ hồ thành con số quyết định được.

👉 **Thực hành:** Bài giải Lab 4 — Docker Compose, Kubernetes & Grafana

SESSION 5

Nghe DevOps: Công việc chính, Chứng chỉ & Lộ trình học

Mục tiêu bài học: Hình dung rõ **công việc thật** của DevOps Engineer, hiểu **vì sao** cần chứng chỉ và giới hạn của nó, và có một **lộ trình học** cụ thể để đi tiếp sau khóa này.

DevOps Engineer thực sự làm gì?

Nhiều người tưởng DevOps Engineer là “người biết dùng nhiều tool”. Thực tế, công việc xoay quanh việc **giữ cho dòng chảy từ code đến người dùng vừa nhanh vừa an toàn**. Một số nhóm việc chính:

- **Vận hành pipeline CI/CD:** sửa build hỏng, tối ưu thời gian chạy, thêm các bước kiểm tra chất lượng và bảo mật.
- **Quản lý hạ tầng cloud:** provisioning bằng IaC (Terraform), tối ưu chi phí (FinOps — một hóa đơn cloud có thể rất đắt nếu không quản lý).
- **Trực on-call & xử lý sự cố:** khi hệ thống có vấn đề, DevOps là người phản ứng. Sau sự cố, viết **post-mortem** — bản phân tích *không đổ lỗi*, tập trung vào “hệ thống nào đã cho phép lỗi này xảy ra”.
- **Bảo mật (DevSecOps):** quét lỗ hổng ngay trong pipeline, quản lý secrets (mật khẩu, API key) an toàn.
- **Cấu nôi Dev – Ops:** viết tài liệu, dựng công cụ nội bộ để developer tự phục vụ (self-service).

🎯 Vì sao post-mortem "không đổ lỗi" lại quan trọng?

Vì nếu văn hóa là trừng phạt người gây lỗi, mọi người sẽ **giấu lỗi** — và bạn mất cơ hội học. Post-mortem tốt hỏi “vì sao hệ thống cho phép một sai lầm đơn lẻ gây sập?” thay vì “ai đã bấm nhầm?”. Con người luôn mắc lỗi; hệ thống tốt phải chịu được lỗi con người. Đây là văn hóa DevOps ở tầng sâu nhất.

DevOps vs SRE vs Platform vs Cloud Engineer

Các vị trí này chồng lấn nhau, gây bối rối. Phân biệt ngắn gọn:

Vai trò	Trọng tâm
DevOps Engineer	Quy trình + tự động hóa CI/CD và hạ tầng

Vai trò	Trọng tâm
SRE (Site Reliability Engineer)	Độ tin cậy hệ thống — SLO, error budget, on-call (khái niệm do Google phổ biến)
Platform Engineer	Xây nền tảng nội bộ để developer tự phục vụ
Cloud Engineer	Chuyên sâu hạ tầng một nhà cung cấp cloud (AWS/Azure/GCP)

Trên thực tế ở nhiều công ty (đặc biệt ở Việt Nam), một người có thể kiêm nhiều vai trò này. Đừng quá bận tâm về nhân — kỹ năng nền tảng chồng lấn rất nhiều.

Kỹ năng nền tảng: vì sao “biết tool” là chưa đủ

Điều nhà tuyển dụng thực sự tìm không phải “đã dùng Kubernetes chưa” mà là **nền tảng đủ vững để tự học bất cứ tool nào**:

- **Linux:** hầu hết server chạy Linux. Không thạo command line thì mọi thứ khác đều khó.
- **Networking:** DNS, HTTP, cổng, firewall, load balancer — hiểu để debug được khi “không kết nối được”.
- **Scripting (Bash/Python):** để tự động hóa những việc chưa có tool sẵn.
- **Một cloud provider (AWS/Azure/GCP):** đi sâu một cái, các cái khác học nhanh.
- **Git:** nền tảng của mọi thứ.
- **Soft skills:** giao tiếp Dev–Ops, viết tài liệu rõ ràng, viết post-mortem. DevOps vốn là bài toán *con người*, không chỉ kỹ thuật.

Vì sao nền tảng quan trọng hơn tool: Tool thay đổi mỗi vài năm (hôm nay Jenkins, mai GitHub Actions). Nền tảng (Linux, mạng, tư duy tự động hóa) thì bền vững hàng chục năm. Đầu tư vào nền tảng cho lãi kép.

Chứng chỉ: vì sao thi, và giới hạn của nó

Vì sao chứng chỉ có giá trị:

- Cho bạn một **lộ trình học có cấu trúc** thay vì học lan man.
- Là **tín hiệu** với nhà tuyển dụng, đặc biệt hữu ích khi bạn chưa có nhiều kinh nghiệm thực chiến.
- Một số (như CKA) thì **thực hành trên terminal thật**, nên chứng minh được kỹ năng thật.

Giới hạn cần nhớ: chứng chỉ chứng minh bạn *học được*, không chứng minh bạn *làm được việc thật*. Nhà tuyển dụng giỏi luôn hỏi thêm về dự án thực tế. Đừng học chỉ để có tấm bằng — học qua **làm dự án thật**, chứng chỉ là hệ quả.

Bản đồ chứng chỉ theo cấp độ

Nền tảng:

- Linux: **LFCS** hoặc **LPIC-1**
- **AWS Certified Cloud Practitioner** (nhập môn cloud)

Trung cấp:

- **AWS Solutions Architect – Associate**
- **CKA – Certified Kubernetes Administrator** (thi thực hành, rất được đánh giá cao)
- **HashiCorp Terraform Associate**

Nâng cao:

- **CKS – Certified Kubernetes Security Specialist**
- **AWS DevOps Engineer – Professional**

Lời khuyên: mỗi giai đoạn chỉ chọn **một** chứng chỉ để tập trung. Học qua dự án thật, dùng chứng chỉ làm cái đích để có động lực và cấu trúc.

Lộ trình học 6–12 tháng

Một lộ trình gợi ý, điều chỉnh theo nền tảng của bạn (dân dev đã biết Git/Docker có thể rút ngắn):

1. **Tháng 1–2 — Nền móng:** Linux, Networking, Git. Thành thạo terminal. *Vì sao trước tiên:* đây là mặt đất mọi thứ khác đứng lên.
2. **Tháng 3–4 — Container & CI/CD:** Docker + dựng pipeline GitHub Actions/GitLab CI. *Vì sao:* đây là kỹ năng tạo giá trị nhìn thấy được sớm nhất.
3. **Tháng 5–6 — Cloud & IaC:** một cloud (AWS) + Terraform; thi Cloud Practitioner. *Vì sao:* gần như mọi việc DevOps ngày nay đều trên cloud.
4. **Tháng 7–9 — Kubernetes & Monitoring:** K8s + Prometheus/Grafana; hướng tới CKA. *Vì sao sau cùng:* K8s khó, cần nền container + cloud trước.
5. **Tháng 10–12 — Dự án & chứng chỉ:** làm một dự án cá nhân hoàn chỉnh, thi một chứng chỉ trung cấp.

💡 Vì sao "làm dự án" quan trọng hơn "xem hết khóa học"?

Kiến thức xem qua sẽ quên. Kiến thức **vật lộn để làm cho chạy** thì ở lại. Mỗi giai đoạn học nên kết thúc bằng một sản phẩm chạy được, và **đưa hết lên GitHub**. Repo đó vừa là nơi ôn tập, vừa là portfolio khi đi xin việc — nhà tuyển dụng tin “code chạy được trên GitHub” hơn dòng “biết Kubernetes” trong CV.

Tài nguyên học tập

- **roadmap.sh/devops** — bản đồ kỹ năng trực quan, cập nhật liên tục.
- **KodeKloud** — nền tảng lab thực hành DevOps rất tốt.
- **Tài liệu chính hãng** — docs.docker.com, kubernetes.io, developer.hashicorp.com. Luôn là nguồn đáng tin nhất.
- **Cộng đồng DevOps Việt Nam** — nơi hỏi đáp, tìm mentor, cập nhật thị trường.

Mini Capstone: khép lại hành trình

Để tổng hợp mọi thứ đã học, hãy thử tình huống:

“Một startup có một web app, đang deploy thủ công bằng FTP, mỗi lần release mất 2 ngày và thường xuyên lỗi.”

Nhiệm vụ: vẽ sơ đồ pipeline DevOps bạn đề xuất, chọn bộ công cụ (và **giải thích vì sao** chọn từng cái — đây mới là phần quan trọng), rồi trình bày.

Và cuối cùng, dùng AI để lập **lộ trình học cá nhân hóa**:

“Tôi là [mô tả nền tảng của bạn], muốn chuyển sang DevOps trong 6 tháng, hãy lập kế hoạch học theo tuần.”

Tóm tắt: những “vì sao” cần nhớ

- **Vì sao DevOps không chỉ là “biết tool”?** Vì cốt lõi là quy trình và văn hóa; tool thay đổi, nền tảng thì bền.
- **Vì sao post-mortem không đổ lỗi?** Để người ta không giấu lỗi, và để sửa hệ thống thay vì trừng phạt con người.
- **Vì sao thi chứng chỉ?** Để có lộ trình có cấu trúc và tín hiệu với nhà tuyển dụng — nhưng phải kèm dự án thật.
- **Vì sao học qua dự án?** Vì kiến thức vật lộn để làm cho chạy mới ở lại, và tạo ra portfolio thật.

 **Bạn đã hoàn thành phần lý thuyết!** Hãy đảm bảo đã làm hết các **bài giải thực hành** — repo của bạn hôm nay chính là dòng đầu tiên trong portfolio DevOps.

SESSION 1

Lab 1 — Dụng môi trường DevOps đầu tiên

Mục tiêu: Có hai môi trường (một máy *dev*, một máy *test*) kết nối được với nhau qua SSH. Đây là nền tảng cho lab Ansible ở Session 2.

Bạn có thể chọn **một** trong hai hướng dưới đây tùy máy của mình. Hướng WSL2 nhẹ hơn, hướng máy ảo (VM) gần với thực tế server hơn.

Lab 1.1 — Môi trường phát triển (dev)

Cách A: Dùng WSL2 (Windows) — nhẹ, nhanh

```
# Mở PowerShell với quyền Administrator, cài Ubuntu qua WSL2
wsl --install -d Ubuntu

# Sau khi khởi động lại và tạo user, kiểm tra:
wsl -l -v
```

Cách B: Dùng máy ảo Ubuntu (VirtualBox/VMware)

1. Tải **Ubuntu Server 22.04 LTS** (file **.iso**).
2. Tạo VM mới trong VirtualBox: 2 CPU, 2–4 GB RAM, 20 GB đĩa.
3. Cài đặt, nhớ **bật OpenSSH server** khi được hỏi trong trình cài đặt.

Cài công cụ cơ bản trên máy dev

```
# Cập nhật hệ thống
sudo apt update && sudo apt upgrade -y

# Cài Git
sudo apt install -y git
git --version

# Cài Docker Engine (script chính thức của Docker)
curl -fsSL https://get.docker.com | sudo sh

# Cho phép chạy docker không cần sudo (đăng xuất/đăng nhập lại sau lệnh này)
sudo usermod -aG docker $USER

# Kiểm tra
docker run hello-world
```

Tạo SSH key và kết nối GitHub

```
# Tạo SSH key (nhấn Enter để dùng mặc định)
ssh-keygen -t ed25519 -C "ban@email.com"

# In public key ra để copy
cat ~/.ssh/id_ed25519.pub
```

Copy nội dung public key, dán vào **GitHub** → **Settings** → **SSH and GPG keys** → **New SSH key**.
Kiểm tra:

```
ssh -T git@github.com
# Kỳ vọng: "Hi <username>! You've successfully authenticated..."
```

💡 Vì sao dùng SSH key thay vì mật khẩu?

SSH key an toàn hơn mật khẩu (không thể đoán/brute-force), và cho phép ****tự động hóa**** — script có thể kết nối server mà không cần gõ mật khẩu. Đây là nền tảng để Ansible hoạt động ở Session 2.

Lab 1.2 — Môi trường kiểm thử (test)

Ta cần một máy thứ hai đóng vai “server đích”. Cách nhẹ nhất: chạy nó như một **container có SSH**.

```
# Trên máy dev, tạo một container Ubuntu chạy SSH server, ánh xạ cổng 2222
docker run -d --name test-server -p 2222:22 rastasheep/ubuntu-sshd:18.04

# Kiểm tra container đang chạy
docker ps
```

Nếu muốn giống thực tế hơn, hãy dựng một **VM Ubuntu thứ hai** thay cho container — các bước SSH bên dưới vẫn tương tự, chỉ đổi cổng và IP.

Kết nối SSH từ máy dev sang máy test

```
# Mật khẩu mặc định của image trên là "root"
ssh root@localhost -p 2222

# Trong phiên SSH, thử vài lệnh rồi thoát
whoami
exit
```

Đưa SSH key sang máy test (để không cần mật khẩu)

```
# Copy public key sang máy test
ssh-copy-id -p 2222 root@localhost

# Giờ kết nối lại — không còn hỏi mật khẩu nữa
ssh root@localhost -p 2222 "echo 'Kết nối không mật khẩu thành công!'"
```

✅ Checkpoint

Bạn đạt yêu cầu của lab khi:

- ☐ Máy dev cài được Git + Docker, chạy `docker run hello-world` thành công.
- ☐ SSH key kết nối được GitHub.
- ☐ SSH từ máy dev sang máy test **không cần mật khẩu**.

AI Prompt để thử

Dán vào ChatGPT/Claude và **so sánh** với những gì bạn vừa làm:

- “Giải thích khác nhau giữa `docker run` và `docker exec` cho người mới.”
- “So sánh Jenkins vs GitLab CI vs GitHub Actions cho team 5 người.”

Hãy tự hỏi: câu trả lời của AI có khớp thực tế bạn vừa trải nghiệm không? Có chỗ nào AI nói chung chung/sai không?

Xử lý lỗi thường gặp

Lỗi	Nguyên nhân & cách sửa
<code>permission denied</code> khi chạy <code>docker</code>	Chưa đăng xuất/đăng nhập lại sau <code>usermod -aG docker</code> . Chạy <code>newgrp docker</code> hoặc mở terminal mới.
<code>Connection refused</code> khi SSH cổng 2222	Container chưa chạy (<code>docker ps</code>), hoặc cổng bị ứng dụng khác chiếm.
<code>ssh-copy-id: command not found</code> (macOS)	Cài <code>brew install ssh-copy-id</code> hoặc copy key thủ công vào <code>~/.ssh/authorized_keys</code> trên máy test.

👉 Quay lại lý thuyết: [Session 1 — Giới thiệu về DevOps](#)

SESSION 2

Lab 2 — Ansible playbook & Terraform

Mục tiêu: Deploy được một web server bằng Ansible playbook, và tạo/quản lý được hạ tầng bằng Terraform. Tận dụng máy test đã dựng ở Lab 1.

Lab 2.1 — Ansible: deploy web bằng một lệnh

Bước 1: Cài Ansible trên máy dev

```
# Ubuntu / WSL
sudo apt update
sudo apt install -y ansible
ansible --version
```

Bước 2: Khai báo inventory (danh sách máy đích)

Tạo file `inventory.ini` trỏ tới máy test (container SSH ở Lab 1, cổng 2222):

```
[web]
test-server ansible_host=localhost ansible_port=2222 ansible_user=root
```

Kiểm tra kết nối:

```
ansible -i inventory.ini web -m ping
# Kỳ vọng: "test-server | SUCCESS => ... "pong"
```

💡 Vì sao chạy `ping` trước?

Module `ping` của Ansible kiểm tra: kết nối SSH có thông không, Python có sẵn trên máy đích không. Luôn chạy nó trước để tách bạch lỗi kết nối khỏi lỗi playbook.

Bước 3: Viết playbook cài Nginx + deploy trang web

Tạo file `deploy-web.yml`:

```
---
- name: Deploy trang web tĩnh bằng Nginx
  hosts: web
  become: yes
  tasks:
    - name: Cài đặt Nginx
```

```

apt:
  name: nginx
  state: present
  update_cache: yes

- name: Tạo trang chủ tùy chỉnh
  copy:
    dest: /var/www/html/index.html
    content: |
      <!doctype html>
      <html lang="vi">
        <head><meta charset="utf-8"><title>DevOps Lab</title></head>
        <body style="font-family:sans-serif;text-align:center;padding:4rem">
          <h1>🚀 Deploy thành công bằng Ansible!</h1>
          <p>Trang này được cài đặt hoàn toàn tự động.</p>
        </body>
      </html>

- name: Đảm bảo Nginx đang chạy và tự bật khi khởi động
  service:
    name: nginx
    state: started
    enabled: yes

```

Bước 4: Chạy playbook — một lệnh, xong

```
ansible-playbook -i inventory.ini deploy-web.yml
```

Kiểm tra kết quả:

```

# Trong container test, Nginx chạy cổng 80 — nhưng ta cần map cổng.
# Nếu dùng container Lab 1, tạo lại container có map thêm cổng 80:
#   docker run -d --name test-server -p 2222:22 -p 8080:80 rastasheep/ubuntu-sshd:18.04
# rồi chạy lại playbook, sau đó mở trình duyệt:
curl http://localhost:8080

```

Điểm mấu chốt cần cảm nhận: Bạn vừa cài phần mềm + deploy web + cấu hình dịch vụ trên một máy từ xa, chỉ bằng **một lệnh, không SSH thủ công**. Nhân con số này lên 300 máy — đó là sức mạnh của Ansible.

Bài tập mở rộng: chứng minh tính idempotent

Chạy lại **đúng lệnh cũ** lần thứ hai:

```
ansible-playbook -i inventory.ini deploy-web.yml
```

Quan sát dòng tổng kết: lần này **changed=0**. Ansible phát hiện mọi thứ đã đúng trạng thái mong muốn nên **không làm gì cả**. Đó chính là idempotent trong thực tế.

Lab 2.2 — Terraform: tạo & quản lý hạ tầng

Ta dùng Terraform để quản lý chính các **Docker container** như hạ tầng (không cần tài khoản cloud).

Bước 1: Cài Terraform

```
# Ubuntu / WSL
wget -O - https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o
/usr/share/keyrings/hashicorp-archive-keyring.gpg
echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg]
https://apt.releases.hashicorp.com $(lsb_release -cs) main" | sudo tee
/etc/apt/sources.list.d/hashicorp.list
sudo apt update && sudo apt install -y terraform
terraform version
```

Bước 2: Viết file hạ tầng `main.tf`

```
terraform {
  required_providers {
    docker = {
      source  = "kreuzwerker/docker"
      version = "~> 3.0"
    }
  }
}

provider "docker" {}

# Tải image nginx
resource "docker_image" "nginx" {
  name          = "nginx:alpine"
  keep_locally = false
}

# Tạo container từ image
resource "docker_container" "web" {
  name = "tf-web"
  image = docker_image.nginx.image_id
  ports {
    internal = 80
    external = 8090
  }
}
```

Bước 3: Vòng đời Terraform

```
# Khởi tạo (tải plugin docker provider)
terraform init

# XEM TRƯỚC những gì sẽ được tạo — chưa làm gì cả
terraform plan
```

```
# Thực sự tạo hạ tầng
terraform apply      # gõ "yes" để xác nhận

# Kiểm tra: mở http://localhost:8090
curl http://localhost:8090
```

Bước 4: Sửa code → apply lại → thấy hạ tầng đổi theo mã

Đổi cổng `external = 8090` thành `8091` trong `main.tf`, rồi:

```
terraform plan      # Terraform báo: sẽ hủy container cũ, tạo container mới ở cổng 8091
terraform apply
```

Điểm mấu chốt: hạ tầng của bạn giờ là **code**. Muốn biết đang chạy gì? Đọc file `.tf`. Muốn tái lập ở máy khác? Copy file, chạy `apply`. Muốn rollback? `git revert` rồi `apply` lại.

Bước 5: Hủy sạch hạ tầng

```
terraform destroy    # gõ "yes" – xóa toàn bộ container Terraform đã tạo
```

✓ Checkpoint

- ☐ `ansible-playbook` deploy được web server; chạy lần hai cho `changed=0`.
- ☐ `terraform apply` tạo được container; sửa `.tf` và `apply` lại thấy hạ tầng thay đổi.
- ☐ `terraform destroy` dọn sạch.

AI Lab (nâng cao)

- Cài một **AI coding agent** (ví dụ Antigravity), yêu cầu: “Viết một Ansible playbook cài Docker + Docker Compose lên Ubuntu.” Đọc kỹ output — nó đúng không? có bước nào thừa/thiếu u?
- Thử chạy **Kubespray** (`git clone https://github.com/kubernetes-sigs/kubespray`) để cài một Kubernetes cluster bằng Ansible. Khi gặp lỗi, **dán log cho AI** và nhờ debug.

Vì sao bài này rèn đúng kỹ năng tương lai: bạn thực hành quy trình “AI viết – người kiểm tra”. Bạn chỉ bắt được lỗi của AI nếu bạn hiểu bản chất Ansible/Terraform ở phần lý thuyết. Công cụ AI khuếch đại người đã hiểu.

👉 Quay lại lý thuyết: [Session 2 — Ansible & Terraform](#)

SESSION 3

Lab 3 — Pipeline CI/CD với GitHub Actions & Docker

Mục tiêu: Mỗi lần push code, ứng dụng tự động được build, test và deploy. Bạn sẽ chứng kiến trọn vòng đời “commit → production”.

Lab 3.1 — Pipeline đầu tiên với GitHub Actions

Bước 1: Chuẩn bị repo ứng dụng mẫu

Tạo một ứng dụng Node.js nhỏ. Cấu trúc thư mục:

```
my-app/  
├─ app.js  
├─ app.test.js  
├─ package.json  
├─ Dockerfile  
└─ .github/  
    └─ workflows/  
        └─ ci-cd.yml
```

`app.js` — một web server đơn giản:

```
const http = require('http');  
  
const handler = (req, res) => {  
  res.writeHead(200, { 'Content-Type': 'text/html; charset=utf-8' });  
  res.end('<h1>🚀 CI/CD hoạt động!</h1>');  
};  
  
// Export để test; chỉ listen khi chạy trực tiếp  
if (require.main === module) {  
  http.createServer(handler).listen(3000, () => console.log('Chạy ở cổng 3000'));  
}  
module.exports = handler;
```

`app.test.js` — một test tối thiểu:

```
const assert = require('node:assert');  
const test = require('node:test');  
const handler = require('./app');  
  
test('handler trả về status 200', () => {  
  let statusCode = 0;  
  const res = {
```

```

    writeHead: (code) => { statusCode = code; },
    end: () => {},
  };
  handler({}, res);
  assert.strictEqual(statusCode, 200);
});

```

`package.json` :

```

{
  "name": "my-app",
  "version": "1.0.0",
  "scripts": {
    "test": "node --test"
  }
}

```

Bước 2: Viết workflow 3 stage (build → test → deploy)

`.github/workflows/ci-cd.yml` :

```

name: CI/CD Pipeline

# Trigger: chạy mỗi khi push lên nhánh main
on:
  push:
    branches: [main]

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - name: Lấy code về
        uses: actions/checkout@v4

      - name: Cài đặt Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'

      - name: Cài dependencies
        run: npm install

      - name: Chạy test (chứa chất lượng)
        run: npm test

  build-docker:
    needs: build-and-test # chỉ chạy nếu test PASS
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Build Docker image
        run: docker build -t my-app:${{ github.sha }} .

```

```
- name: (Demo) In ra image đã build
  run: docker images | grep my-app
```

Bước 3: Push và quan sát

```
cd my-app
git init
git add .
git commit -m "Thêm ứng dụng và pipeline CI/CD"
git branch -M main
git remote add origin git@github.com:<username>/my-app.git
git push -u origin main
```

Mở repo trên GitHub → tab **Actions**. Bạn sẽ thấy pipeline tự chạy: **build-and-test** → **build-docker**.

Vì sao **needs: build-and-test** quan trọng?

Nó tạo **cửa chốt lượng**: job build Docker chỉ chạy nếu test đã PASS. Code lỗi sẽ bị chặn ngay, không bao giờ được đóng gói đem deploy. Đây chính là "CI" trong thực tế — máy tự bảo vệ nhánh main khỏi code hỏng.

Bài tập: xem cửa chốt lượng hoạt động

Cố tình sửa **app.js** cho test fail (ví dụ đổi **200** thành **500**), push lên. Quan sát: pipeline **dừng ở stage test**, job Docker **không chạy**. Sửa lại, push, pipeline xanh trở lại.

Lab 3.2 — CI/CD với Docker: build image & deploy

Dockerfile

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install --production
COPY . .
EXPOSE 3000
CMD ["node", "app.js"]
```

Thêm stage deploy (đẩy image lên registry + deploy máy test)

Mở rộng workflow — đẩy image lên **GitHub Container Registry**, rồi deploy qua SSH sang máy test:

```
deploy:
  needs: build-docker
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
```

```

- name: Đăng nhập GitHub Container Registry
  run: echo "${{ secrets.GITHUB_TOKEN }}" | docker login ghcr.io -u ${{ github.actor }} --password-stdin

- name: Build & push image
  run: |
    docker build -t ghcr.io/${{ github.actor }}/my-app:latest .
    docker push ghcr.io/${{ github.actor }}/my-app:latest

# Deploy sang máy test qua SSH (dùng playbook Ansible từ Session 2!)
- name: Deploy qua SSH
  uses: appleboy/ssh-action@v1.0.0
  with:
    host: ${{ secrets.SERVER_HOST }}
    username: ${{ secrets.SERVER_USER }}
    key: ${{ secrets.SSH_PRIVATE_KEY }}
    script: |
      docker pull ghcr.io/${{ github.actor }}/my-app:latest
      docker stop my-app || true
      docker rm my-app || true
      docker run -d --name my-app -p 80:3000 ghcr.io/${{ github.actor }}/my-app:latest

```

⚠ Vì sao dùng `secrets` , không viết thẳng mật khẩu?

Không bao giờ đặt mật khẩu/SSH key trực tiếp trong file YAML — nó nằm trong Git, ai xem repo cũng thấy. Dùng **GitHub Secrets** (Settings → Secrets and variables → Actions). Đây là bài học bảo mật (DevSecOps) cơ bản nhất.

Nội liên kiến thức: để ý bước deploy dùng đúng cơ chế SSH bạn đã thiết lập ở Lab 1, và có thể thay bằng chính Ansible playbook ở Lab 2. Toàn bộ khóa học là một dự án xuyên suốt.

Bonus: Jenkins để so sánh

```

# Chạy Jenkins bằng Docker
docker run -d --name jenkins -p 8080:8080 -p 50000:50000 \
  -v jenkins_home:/var/jenkins_home jenkins/jenkins:lts

# Lấy mật khẩu admin ban đầu
docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword

```

Mở <http://localhost:8080> , tạo một freestyle job đơn giản. So sánh trải nghiệm: Jenkins mạnh và linh hoạt nhưng phải tự host & bảo trì; GitHub Actions tích hợp sẵn, nhẹ để bắt đầu.

✓ Checkpoint

- ☐ Push code → pipeline tự chạy build → test → (Docker).

- ☐ Test fail thì pipeline dừng, không deploy code lỗi.
- ☐ (Nâng cao) Deploy được image lên máy test, mở trình duyệt thấy app chạy.

AI Prompt để thử

- Dán file `ci-cd.yml` vào AI: “Review pipeline này, chỉ ra lỗi hỏng bảo mật và điểm có thể tối ưu.”
- “Sinh thêm unit test cho `app.js` để tăng độ phủ.”

Hãy kiểm chứng: đề xuất của AI có hợp lý với ngữ cảnh của bạn không?

👉 Quay lại lý thuyết: [Session 3 — CI/CD](#)

SESSION 4

Lab 4 — Docker Compose, Kubernetes & Grafana

Mục tiêu: Chạy ứng dụng trên Kubernetes local và xem được metrics trên Grafana. Trải nghiệm self-healing của K8s.

Lab 4.1 — Docker Compose (ứng dụng nhiều service)

Tạo file `docker-compose.yml` chạy web + database:

```
services:
  web:
    image: nginx:alpine
    ports:
      - "8080:80"
    depends_on:
      - db

  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_PASSWORD: secret
      POSTGRES_DB: appdb
    volumes:
      - db-data:/var/lib/postgresql/data

volumes:
  db-data:
```

```
# Dựng toàn bộ ứng dụng
docker compose up -d

# Xem các service đang chạy
docker compose ps

# Kiểm tra web
curl http://localhost:8080

# Kiểm tra network: web ping tới db được không
docker compose exec web ping -c 2 db
```

🎯 Vì sao `db` ping được bằng tên, không cần IP?

Docker Compose tự tạo một mạng nội bộ và **DNS** cho các service. Container `web` gọi `db` bằng đúng tên service. Đây là nền tảng để hiểu Service trong Kubernetes ở phần sau — cùng một ý tưởng "địa chỉ ổn định bằng tên".

```
# Dọn dẹp
docker compose down          # giữ lại volume dữ liệu
# docker compose down -v     # xóa cả volume
```

Lab 4.2 — Kubernetes với Minikube

Bước 1: Cài Minikube + kubectl

```
# Minikube (Linux)
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube

# kubectl
curl -LO "https://dl.k8s.io/release/$(curl -L -s
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
sudo install kubectl /usr/local/bin/kubectl

# Khởi động cluster local
minikube start
kubectl get nodes    # thấy 1 node "Ready"
```

Bước 2: Viết manifest (Deployment + Service)

`app.yaml` :

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 1          # bắt đầu với 1 bản
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: web
          image: nginx:alpine
          ports:
```

```

- containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: web-service
spec:
  type: NodePort
  selector:
    app: web
  ports:
    - port: 80
      targetPort: 80

```

Bước 3: Deploy và truy cập

```

kubectl apply -f app.yaml

kubectl get pods          # thấy 1 pod "Running"
kubectl get deployments
kubectl get services

# Mở app trong trình duyệt (Minikube tạo URL)
minikube service web-service --url

```

Bước 4: Scale 1 → 3 replica

```

kubectl scale deployment web --replicas=3
kubectl get pods          # giờ thấy 3 pod

```

Bước 5: Chứng kiến self-healing 🔥

Đây là phần “aha” của lab. Xóa một pod và quan sát:

```

# Lấy tên một pod
kubectl get pods

# Xóa nó đi
kubectl delete pod <tên-pod>

# Ngay lập tức xem lại
kubectl get pods

```

Bạn sẽ thấy Kubernetes **tự tạo một pod mới** thay thế trong vài giây — vì bạn đã khai báo “muốn 3 replica”, K8s liên tục ép hệ thống về đúng trạng thái đó.

Vì sao đây là bước ngoặt tư duy: Bạn không “quản lý pod”, bạn khai báo *ý muốn*. Pod chết, node sập — không quan trọng; K8s tự đưa hệ thống về đúng thứ bạn muốn. Đây là lý do bạn không cần trực đêm để bật lại dịch vụ.

Lab 4.3 — Monitoring nhanh (Prometheus + Grafana)

Cách nhanh nhất: chạy bằng Docker Compose. Tạo `monitoring/docker-compose.yml`:

```
services:
  prometheus:
    image: prom/prometheus:latest
    ports:
      - "9090:9090"
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml

  grafana:
    image: grafana/grafana:latest
    ports:
      - "3000:3000"
    environment:
      GF_SECURITY_ADMIN_PASSWORD: admin

  node-exporter:
    image: prom/node-exporter:latest
    ports:
      - "9100:9100"
```

`monitoring/prometheus.yml` — cấu hình Prometheus thu thập metrics:

```
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: 'node'
    static_configs:
      - targets: ['node-exporter:9100']
```

```
cd monitoring
docker compose up -d
```

- Mở **Prometheus**: `http://localhost:9090` → thử query `up` (xem target nào đang sống).
- Mở **Grafana**: `http://localhost:3000` (admin / admin).
 1. Add data source → Prometheus → URL `http://prometheus:9090`.
 2. Import dashboard mẫu: **Dashboards** → **Import** → nhập ID `1860` (Node Exporter Full) → chọn data source Prometheus.

3. Xem CPU / RAM / mạng của hệ thống hiển thị trực quan.

Vì sao bước này khép lại vòng lặp DevOps: bạn đã đi hết Plan → Code → Build → Test → Deploy, và giờ là **Monitor**. Nhìn dashboard, bạn “thấy” hệ thống đang sống — đây là thứ biến “deploy và cấu nguyên” thành “deploy và quan sát”.

✓ Checkpoint

- ☐ Compose chạy web + db, hai container nói chuyện được với nhau.
- ☐ App chạy trên Kubernetes; scale lên 3 replica.
- ☐ Xóa 1 pod, thấy K8s tự tạo pod mới (self-healing).
- ☐ Grafana hiển thị metrics CPU/RAM.

AI Prompt để thử

Cố tình gây lỗi để tạo pod `CrashLoopBackOff` (ví dụ đặt `image: nginx:phien-ban-khong-ton-tai`), rồi:

```
kubectl describe pod <tên-pod>    # xem sự kiện lỗi
kubectl logs <tên-pod>           # xem log
```

Dán output lỗi vào AI: “Pod của tôi bị lỗi này, nguyên nhân và cách sửa?” — So sánh chẩn đoán của AI với những gì `kubectl describe` cho thấy. AI nhanh hơn hay chính xác hơn?

Xử lý lỗi thường gặp

Lỗi	Cách xử lý
<code>minikube start</code> treo	Kiểm tra driver: <code>minikube start --driver=docker</code> . Cần Docker đang chạy.
Pod <code>ImagePullBackOff</code>	Sai tên image hoặc không tải được. <code>kubectl describe pod</code> để xem chi tiết.
Grafana không thấy data	Data source URL phải là <code>http://prometheus:9090</code> (tên service), không phải <code>localhost</code> .

👉 Quay lại lý thuyết: [Session 4 — Container, Kubernetes & Giám sát](#)